

Знакомство с OpenMP

OpenMP

Стандарт интерфейса для многопоточного программирования над общей памятью

Набор средств для языков C/C++ и Fortran:

Директивы компилятора

`#pragma omp ...`

Библиотечные подпрограммы (функции)

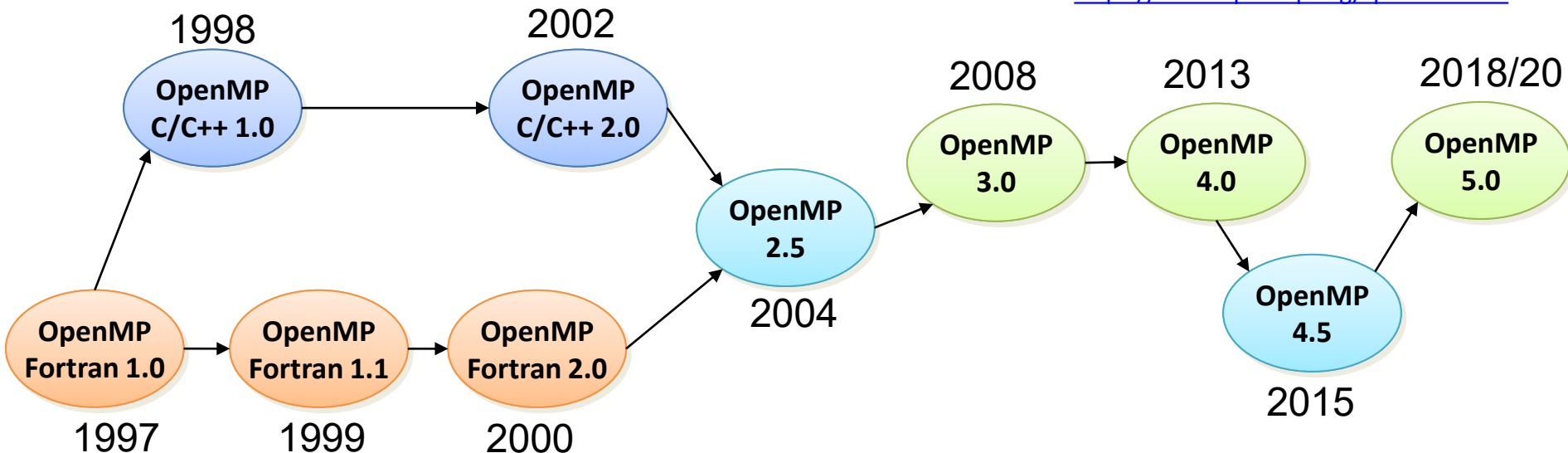
`get_num_threads()`

Переменные окружения

`OMP_NUM_THREADS`

История:

<https://www.openmp.org/specifications>



Поддержка компиляции OpenMP

Стандарт OpenMP	Компиляторы (начальные версии)
2.0	Visual C++, gcc, Intel, AMD, IBM XL, и множество других компиляторов
3.0	GCC 4.3.1, Intel 12.0, MSVC 2022 17.2 (частично с особым ключом)
4.0	GCC 4.9.0, Intel 15.0, LLVM/Clang 3,9 MSVC 2022 17.7 (экспериментально omp_llvm.h)
4.5	GCC 6, Intel compilers 17.0, LLVM/Clang 7
5.0	GCC 9 (частично), 12 полностью Intel 19.1 (частично), 2021 (полностью) LLVM/Clang 8 (частично), 11 (полностью)

Модель с разделяемой памятью

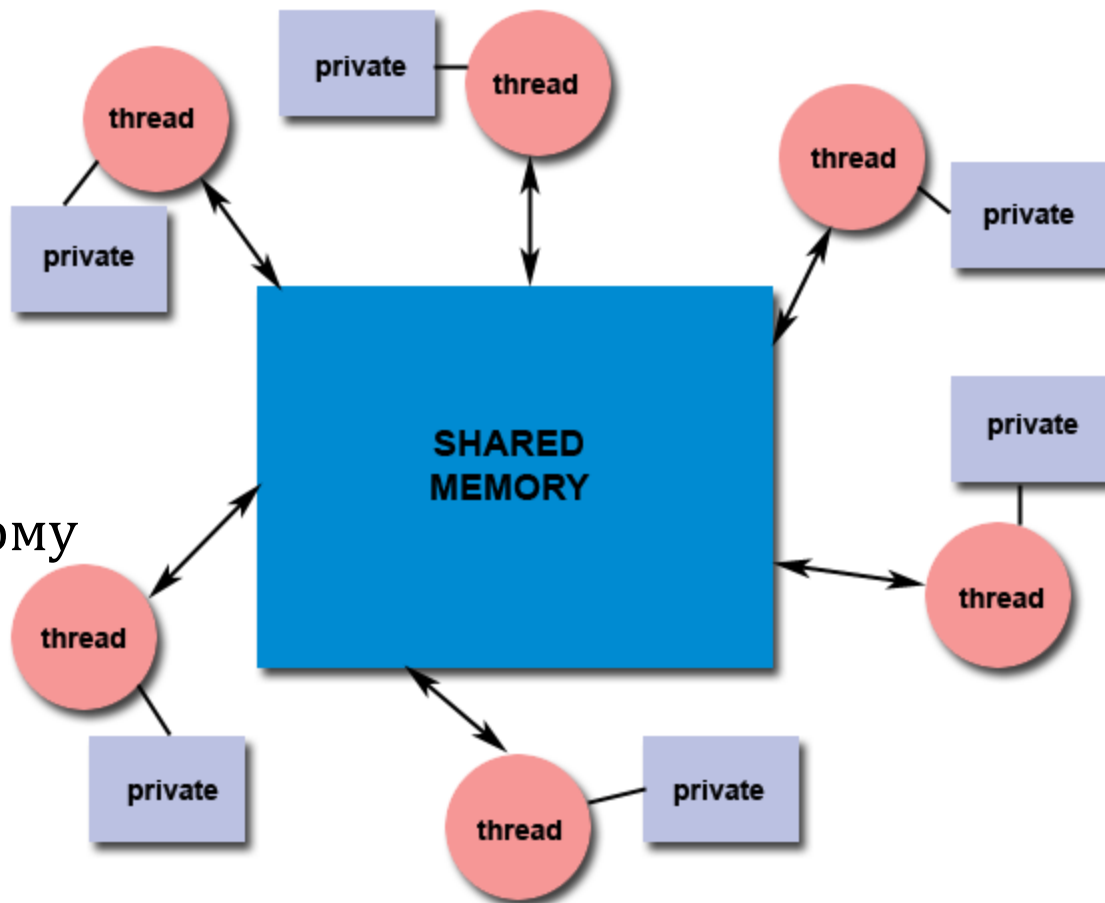
Все потоки имеют доступ к глобальной общей (разделяемой) памяти

Данные могут быть **разделяемые (общие)** и **приватные**

➤ Разделяемые данные доступны всем потокам

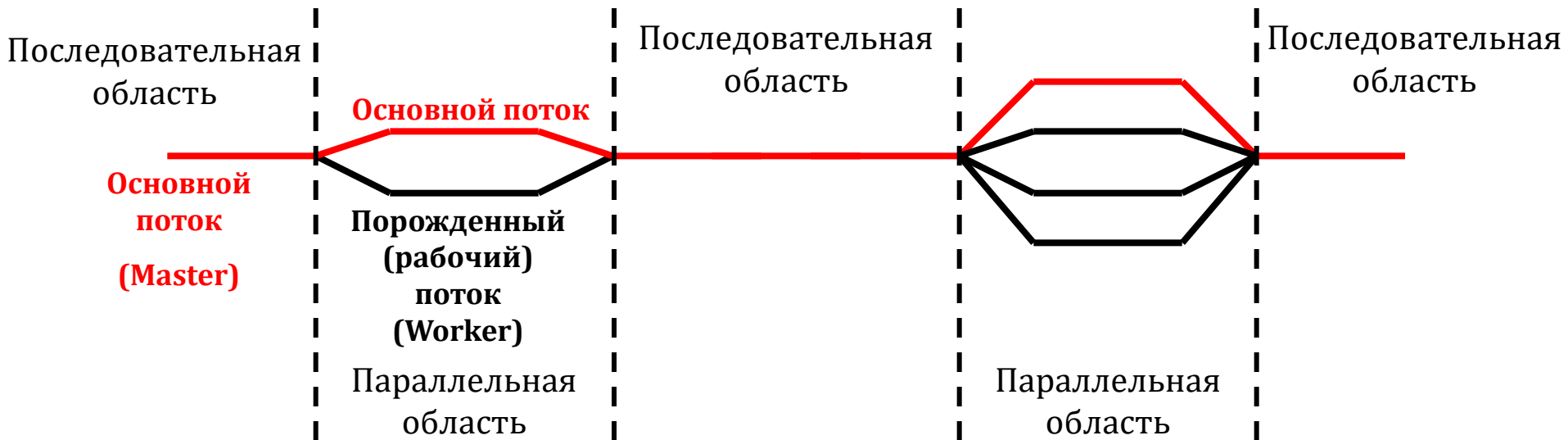
➤ Приватные — только одному

Синхронизация требуется
Для доступа к общим
данным



Модель программирования

Fork-join («разветвление-соединение») программирование:
Основной поток порождает группу дополнительных потоков
В конце параллельной области все потоки соединяются в один



- ☐ Явное указание параллельных секций
- ☐ Поддержка вложенного параллелизма
- ☐ Поддержка динамических потоков

Формат директив, переменные окружения

C/C++: регистр имеет значение, синтаксис:

#pragma omp название-директивы[опция [[,] опция]...]
{
 структурный блок
}

❖ Продление: использовать «\» в прагме

Переменные окружения

Название	Описание
OMP_NUM_THREADS n	Устанавливает количество используемых потоков в параллельной области программы
OMP_SCHEDULE “schedule,[chunk]”	Устанавливает способ планирования и размер порции при выполнении циклов.
OMP_DYNAMIC { TRUE FALSE }	Разрешает или запрещает динамическое регулирование числом потоков
OMP_NESTED { TRUE FALSE }	Устанавливает или отменяет вложенный режим параллельной обработки.

Функции OpenMP

Название	Описание
<code>void omp_set_num_threads (int num_threads);</code>	Установить количество создаваемых потоков
<code>int omp_get_max_threads (void);</code>	Получение максимально возможного количества потоков
<code>int omp_get_num_threads (void);</code>	Получение количества потоков в параллельной области программы
<code>int omp_get_thread_num (void);</code>	Получение номера потока
<code>int omp_get_num_procs (void);</code>	Получение числа вычислительных элементов (процессоров или ядер), доступных приложению
<code>void omp_set_dynamic (int dynamic);</code>	Установить режим динамического создания потоков
<code>int omp_get_dynamic (void);</code>	Получение состояние динамического режима
<code>void omp_set_nested (int nested);</code>	Установить режим поддержки вложенных параллельных фрагментов
<code>int omp_get_nested (void);</code>	Получения состояние режима поддержки вложенных параллельных фрагментов
<code>double omp_get_wtime (void);</code>	Получение времени текущего момента выполнения программы
<code>double omp_get_wtick (void);</code>	Получение времени в секундах между двумя последовательными показателями времени аппаратного таймера
<code>int omp_in_parallel (void);</code>	Проверка нахождения программы в параллельном фрагменте

Структурный блок

Действие директив распространяется на структурный блок:

#pragma omp название-директивы[опция [[,] опция]...]

```
{  
    структурный блок  
}
```

Структурный блок — блок кода с одной точкой входа и одной точкой выхода.

```
#pragma omp parallel  
{  
    ...  
    mainloop: res[id] = f(id);  
    if (res[id] != 0)  
        goto mainloop;  
    ...  
    exit (0);  
}
```

Структурный блок

```
#pragma omp parallel  
{  
    ...  
    mainloop: res[id] = f(id);  
    ...  
}  
if (res[id] != 0) goto mainloop;
```

Неструктурный блок

Объявление параллельного блока

Команда **parallel** – основная директива OpenMP. Синтаксис:

```
#pragma omp parallel [ опция [ [,] опция ]...]
```

```
{  
    структурный блок  
}
```

По-умолчанию, создается ровно столько потоков, сколько доступно ядер/процессоров. Потоки пронумерованы от 0 до N, где 0 – основной

Условие использования параллельной обработки

Используется опция **if**

```
#pragma omp название-директивы if (скалярное выражение)
```

- Исполнить параллельно, если выражение истинно
- В противном случае – последовательно

Изменение количества используемых потоков

1. Опция **num_threads** команды **parallel**

#pragma omp parallel num_threads (целое_значение)

```
#pragma omp parallel num_threads (4)
{
    . . .
}
```

2. Функция **omp_set_num_threads**

(требуется подключение заголовочного файла **omp.h**)

void omp_set_num_threads (int Количество_потоков);

```
omp_set_num_threads (4) ;
#pragma omp parallel
{
    . . .
}
```

3. Переменная окружения **OMP_NUM_THREADS**

set OMP_NUM_THREADS = целое_значение

Пример программы «Hello, World!»

```
#include <omp.h>
```

```
...
```

```
int main()
```

```
{
```

```
    printf("Hello, World!\n");
```

```
#pragma omp parallel
```

```
{
```

```
    int i, n;
```

```
    i = omp_get_thread_num();
```

```
    n = omp_get_num_threads();
```

```
    printf("I'm thread %d of %d\n", i, n);
```

```
}
```

```
return 0;
```

```
}
```

Пример переназначения количества потоков в OpenMP

```
int main()
{
    int max_a_threads = 0, a_treads = 0;
    max_a_threads = omp_get_max_threads();
    printf("Default threads %d \n", max_a_threads );

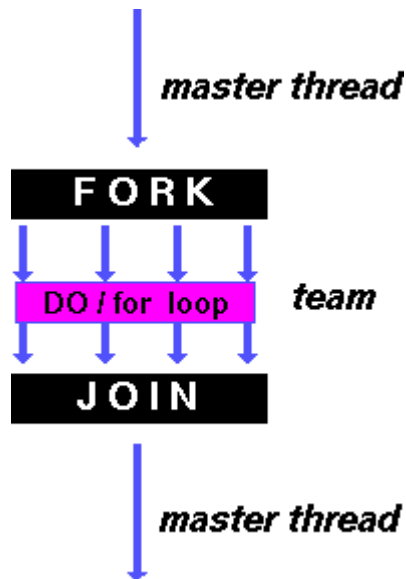
    omp_set_num_threads(4);
    a_treads = omp_get_max_threads();
    printf("Available threads %d \n", a_treads);

    int new_threads = 3;
    omp_set_num_threads(new_threads);
    a_treads = omp_get_max_threads();
    printf("Available threads %d \n", a_treads);

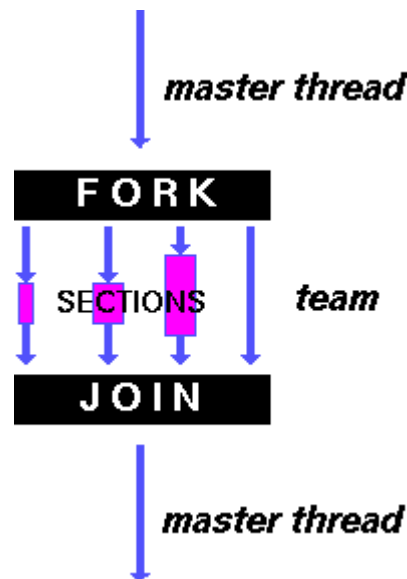
    return 0;
}
```

Способы разделения работы между потоками

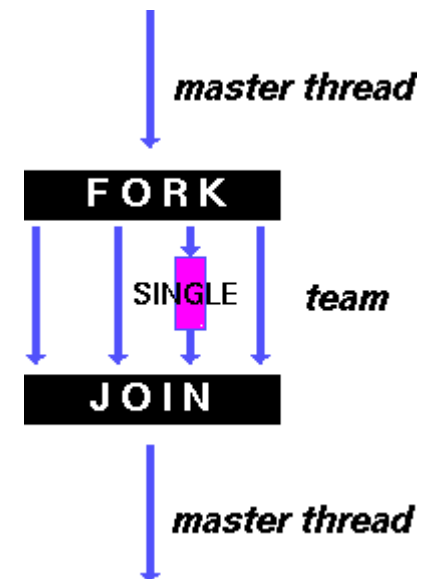
```
#pragma omp for
for (i=0;i<N;i++)
{
    // Код
}
```



```
#pragma omp sections
{
    #pragma omp section
    // Код задачи 1
    #pragma omp section
    // Код задачи 2
}
```



```
#pragma omp single
{
    // Код
}
```



Пример: Директива **omp for**

```
#include <omp.h>
...

int main()
{
    int i;

    #pragma omp parallel
    {
        #pragma omp for
            for (i=0; i<1000; i++)
                printf("%d ", i);
    }
    return 0;
}
```

```
#include <omp.h>
...

int main()
{
    int i;

    #pragma omp parallel for
        for (i=0; i<1000; i++)
            printf("%d ", i);

    return 0;
}
```

Пример: Директива **omp sections**

```
int main()
{
#pragma omp parallel sections
{
#pragma omp section
{
    printf("1st half\n");
    for (int i = 0; i < 500; i++)
        printf("%d ", i);
}
#pragma omp section
{
    printf("2nd half\n");
    for (int i = 501; i < 1000; i++)
        printf("%d ", i);
}
}
return 0;
}
```

Ручная равномерная балансировка нагрузки для директивы **omp sections**

$$S_{start_i} = \frac{(Section_N - 1) \times Size}{p}$$

$$S_{end_i} = \frac{(Section_N) \times Size}{p}$$

Пример расчета для 4х секций

$S_0=0$

$S_1=(1 \times Size)/p$

$S_2=(2 \times Size)/p$

$S_3=(3 \times Size)/p$

$S_4=(4 \times Size)/p$

При запуске работы в секциях нужно проверять допустимые потоки **p** чтобы не выйти за границу массива.

Пример значения S_* для обработки 1000000 элементов

переменные	p=2	P=3	P=4
S_1	500000	333333	250000
S_2	1000000	666666	500000
S_3	1500000	1000000	750000
S_4	2000000	1333333	1000000

Ручная равномерная балансировка нагрузки для директивы **omp sections**

```
#pragma omp parallel sections
{
#pragma omp section
{
    do_work(Array, S_0, S_1);
}
#pragma omp section
{
    if(p>1) do_work(Array, S_1, S_2);
}
#pragma omp section
{
    if(p>2) do_work(Array, S_2, S_3);
}
#pragma omp section
{
    if(p>3) do_work(Array, S_3, S_4);
}
}
```

Области видимости переменных

Переменные, объявленные внутри параллельного блока, являются локальными для потока:

```
#pragma omp parallel
{
    int num;
    num = omp_get_thread_num();
    printf("Поток % d\n", num);
}
```

Переменные, объявленные вне параллельного блока, определяются параметрами директив OpenMP:

- | | |
|---------------------------------------|--|
| ☐ private | ☐ default |
| ☐ firstprivate | ☐ reduction |
| ☐ lastprivate | ☐ threadprivate |
| ☐ shared | ☐ copying |

Области видимости переменных

Переменные, объявленные вне параллельного блока, определяются параметрами директив OpenMP:

➤ *private*

- firstprivate
- lastprivate
- shared
- default
- reduction
- threadprivate
- copying

Своя локальная переменная в каждом потоке

```
int num;  
#pragma omp parallel private(num)  
{  
    num = omp_get_thread_num();  
    printf("%d\n", num);  
}
```

Конфликт доступа к данным

```
#define N 50

int main()
{
    float a[N], tmp;
    // Фрагмент кода ...
    #pragma omp parallel
    {
        #pragma omp for
        for(int i=0; i<N; i++)
        {
            tmp = a[i] * a[i];
            a[i] = 1 - tmp;
        }
    }
    // Фрагмент кода ...
    return 0;
}
```

Ошибка возникает при одновременном выполнении следующих условий:

- ❑ Два или более потока обращаются к одной и той же ячейке памяти.
- ❑ По крайней мере, один из этих доступов к памяти является записью.
- ❑ Потоки не синхронизируют свой доступ к данной ячейке памяти.

При одновременном выполнении всех трех условий порядок доступа становится неопределенным.

Конфликт доступа к данным

```
#define N 50
```

```
int main()
{
    float a[N], tmp;
    // Фрагмент кода ...
    #pragma omp parallel
    {
        #pragma omp for
        for(int i=0; i<N; i++)
        {
            tmp = a[i] * a[i];
            a[i] = 1 - tmp;
        }
    }
    // Фрагмент кода ...
    return 0;
}
```

```
#define N 50
```

```
int main()
{
    float a[N], tmp;
    // Фрагмент кода ...
    #pragma omp parallel
    {
        #pragma omp for private(tmp)
        for(int i=0; i<N; i++)
        {
            tmp = a[i] * a[i];
            a[i] = 1 - tmp;
        }
    }
    // Фрагмент кода ...
    return 0;
}
```

Области видимости переменных

Переменные, объявленные вне параллельного блока, определяются параметрами директив OpenMP:

- private

- *firstprivate*

- lastprivate

- shared

- default

- reduction

- threadprivate

- copying

Локальная переменная с инициализацией

```
int num=5;
#pragma omp parallel \
    firstprivate(num)
{
    printf("%d\n", num);
}
```

Области видимости переменных

Переменные, объявленные вне параллельного блока, определяются параметрами директив OpenMP:

- private
- firstprivate
- *lastprivate*
- shared
- default
- reduction
- threadprivate
- copying

**Локальная переменная с сохранением
последнего значения
(в последовательном исполнении)**

```
int i, j;  
#pragma omp parallel for \  
                    lastprivate(j)  
    for (i=0; i<100; i++) j=i;  
printf("Последний j = %d\n", j);
```

Области видимости переменных

Переменные, объявленные вне параллельного блока, определяются параметрами директив OpenMP:

- private
- firstprivate
- lastprivate
- *shared*
- default
- reduction
- threadprivate
- copying

Разделяемая (общая) переменная

```
int j;  
  
#pragma omp parallel for \  
    shared(j)  
for (int i=0; i<100; i++) j=i;  
  
printf("j = %d\n", j);
```


Области видимости переменных

Переменные, объявленные вне параллельного блока, определяются параметрами директив OpenMP:

- private
- firstprivate
- lastprivate
- shared
- *default*
- reduction
- threadprivate
- copying

Задание области видимости не указанных явно переменных [default(shared | none)]

```
int i, k, n=2, a[20];
#pragma omp parallel private(i,k) \
                        default(shared)
{
    i = omp_get_thread_num() / n;
    k = omp_get_thread_num() % n;
    a[omp_get_thread_num()]=i;
    printf("%d %d %d\n", i, k, n);
}
```

Области видимости переменных

Переменные, объявленные вне параллельного блока, определяются параметрами директив OpenMP:

- private
- firstprivate
- lastprivate
- shared
- default
- *reduction*
- threadprivate
- copying

**Переменная для выполнения
редукционной операции**

```
int i, s=0;
#pragma omp parallel for \
                    reduction(+:s)
    for (i=0;i<100;i++)
        s += i;

printf("Sum = %d\n", s);
```

В качестве редукционной операции могут выступать:

арифметические или логических операции (+, -, *, &, |, ^, && ||)

функции (**max**, **min**) , которые доступны с **OpenMP 3.1**

Области видимости переменных

Переменные, объявленные вне параллельного блока, определяются параметрами директив OpenMP:

- private
- firstprivate
- lastprivate
- shared
- default
- reduction
- *threadprivate*
- copying

**Объявление глобальных переменных
локальными для потоков**

```
int x;  
#pragma omp threadprivate(x)  
  
int main()  
{  
    . . .  
}
```

Области видимости переменных

Переменные, объявленные вне параллельного блока, определяются параметрами директив OpenMP:

- private
- firstprivate
- lastprivate
- shared
- default
- reduction
- threadprivate

➤ *copying*

**Объявление глобальных переменных
локальными для потоков с
инициализацией**

```
int x;  
#pragma omp threadprivate(x)  
#pragma omp copyin(x)
```

```
int main()  
{  
    . . .  
}
```

Способ синхронизации данных

Синхронизация типа **critical**

Синхронизация типа **critical** используется для описания структурных блоков, выполняющихся только в одном потоке из всего набора параллельных потоков.

В программах, написанных на языке C/C++, используется прагма (где **name** - имя критической секции):

```
#pragma omp critical [ name ]  
{<структурный блок программы>}
```

Разные критические секции независимы, если они имеют разные имена. Не поименованные критические секции относятся к одной и той же секции.

- ❑ Полезны для избавления от ошибок соревнования, чтения записи данных (неопределенный порядок)
- ❑ Может привести к тому, что параллельная программа станет последовательной

Способ синхронизации данных

Синхронизация типа **critical** (пример)

Моделирование очереди, в которой задание выбирается из очереди и обрабатывается. Для защиты от многократной выборки потоками из очереди одного и того же задания операция выборки должна выполняться в критической секции. Т.к. две очереди в этом примере идентичны, то они защищены критическими секциями с разными именами: **xaxis** и **yaxis**.

```
. . .  
#pragma omp parallel shared(x, y) private(x_next, y_next)  
{  
    #pragma omp critical ( xaxis )  
        x_next = dequeue(x);  
        work(x_next);  
    #pragma omp critical ( yaxis )  
        y_next = dequeue(y);  
        work(y_next);  
}  
. . .
```

Совместимость директив и их параметров (опция)

Параметр (опция)	Директива					
	parallel	for	sections	single	parallel for	parallel sections
if	❖				❖	❖
private	❖	❖	❖	❖	❖	❖
firstprivate	❖	❖	❖	❖	❖	❖
lastprivate		❖	❖		❖	❖
shared	❖	❖			❖	❖
default	❖				❖	❖
reduction	❖	❖	❖		❖	❖
copying	❖				❖	❖
shedule		❖			❖	
ordered		❖			❖	
nowait		❖	❖	❖		